

LAB02: EXPRESSIONS & FUNCTIONS

CONTENT

Objective	2
CORE Guidelines & Naming conventions	2
naming conventions	2
Core guidelines	2
Exercises	3
Ex01: Explore Random	3
general	3
function to print separating line	3
print 4 random numbers	4
print random numbers in interval	4
print random floats in interval	4
Ex02: Explore strings	4
GENERAL	4
ascii values	5
email generator (combining strings and characters)	5
rhyme generator (converting numbers to strings)	5
combining integers and strings	6
Ex03: Explore math	6
GENERAL	6
print a test table	6
Ex04: Awaits human review	7
Ex03: HANDS-ON DEMO: using trigonometry to draw	7
Ex05: drawing with basic shapes	8
GENERAL	8
Make the 6 drawings	8
Maintainable and readable code	9
Ex06: Random Rectangles	10
Ex07: Beam	10
Restrictions	11
Change the window size	11
Ex08: Coding Craftsmanship Challenge	11
Instructions	Error! Bookmark not defined.
Wall of Fame	11
Submission instructions	12

Before zipping	12
Submit	12
References	13
Structure of a program	13
cout, cin, endl, std, using	13
Literals	13
Operator precedence and associativity	13
Compiling and linking	13

OBJECTIVE

At the end of these exercises, you should be able to:

- Explain what the Visual Studio build does and how to solve the reported errors
- Define, initialize and manipulate the `std::string` type
- Convert a string containing a number to a numeric value and vice versa
- Define and use simple functions
- Use existing functions that calculate a result: `cos`, `sin`, `sqrt`...
- Find and solve errors and warnings related to the above topics
- Generate random numbers in given ranges
- Get familiar with the modulo operator
- Typecast types to other types, the integer division
- Explain the difference between the prefix and postfix increment/decrement
- Avoid magic/hard coded numbers that make your code less readable and hard to maintain
- Draw using transparent colors

We advise you to **make your own summary of topics** which are new to you.

CORE GUIDELINES & NAMING CONVENTIONS

NAMING CONVENTIONS

The **naming conventions** used in Prog1 and Prog2 promote a consistent coding style and should be followed, just as you will need to follow the conventions of any project, team, or company you end up with.

Category	Naming Convention / Pattern	Code Example
Local Variables	lowerCamelCasing	<code>float width{};</code> <code>const float innerSize{ 10.f };</code>
Boolean Identifiers	Verb indicating boolean state (Is..., Has...)	<code>bool isReady{};</code> <code>bool IsKeyPressed();</code>
Functions / Methods	UpperCamelCasing (Verb-based)	<code>ClearBackground(); // return type void</code>

CORE GUIDELINES

Always check the **PROG1: Naming Conventions & C++ Core Guidelines** document for a detailed explanation about the guidelines below.
(**New guidelines** introduced in this lab are indicated in **blue**)

- ✓ **F1;** Name meaningful operations
- ✓ **F3;** Keep functions short and simple
- ✓ **ES. 10;** Declare one name (only) per declaration
- ✓ **ES. 20;** Always initialize an object
- ✓ **ES.23;** Prefer the {} initializer syntax
- ✓ **ES.48;** Avoid casts if possible
- ✓ **NL.1;** Don't say in comments what the code already says
- ✓ **NL.3;** Keep comments crisp
- ✓ **NL.5;** Don't encode type information in names
- ✓ **NL.8;** Use a consistent naming style
- ✓ **NL.20;** Don't place two statements on the same line
- ✓ **NR.1;** Don't force all declarations to the top

EXERCISES

Each lab you should **create a folder** with the correct name:

- **L02_Name_FirstName**
 - Replace **Name** and **FirstName** by your personal last and first name
 - Avoid any special characters like ` or - by replacing them with an underscore _
 - Replace **L02** with the correct lab number each week
 - All projects from this lab should be created inside this folder

EX01: EXPLORE RANDOM

GENERAL

- Create a new **CONSOLE PROJECT** called **EX01_ExploreRandom** in this lab's folder
- Topics needed for this exercise: creating functions, rand(), srand()

FUNCTION TO PRINT SEPARATING LINE

-- Before you start, make sure you remember the 3 steps that need to be taken to define, implement and use a function. Check theory if necessary. --

- create a function to draw a line to the console that will separate the different parts in the console
 - it should add **two new lines** in the console, followed by a some "=", then again **2 new lines**
- choose a good name that follows the naming conventions
- call the function 3 times to get a result similar to this:

```
=====
=====
=====
```

- remember to call this function between each part of the exercise here

-- Do not forget that from now on, we **always** create functions for the separate parts of the assignments! --

PRINT 4 RANDOM NUMBERS

-- We will use the [rand\(\)](#) function to generate pseudo random numbers. --

- Create a function that will show 4 random numbers in the output window; choose a good name.
- Print a random number 4 times by executing `std::cout << rand() << std::endl;`



```
=====
41
18467
6334
26500
=====
```

- Check the above screenshot. Start and stop the application a few times; the numbers are not very random at all! Remember how to fix this?
 - Right, we need to *seed* the application.
 - How: `srand( static_cast<unsigned int>( time( nullptr ) ) );`
 - Add this at the start of your **main**; it should only be executed once.

PRINT RANDOM NUMBERS IN INTERVAL

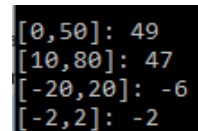
-- We will use the [rand\(\)](#) function combined with the **modulo operator %**. --

- Create and call a second function that generates 4 random numbers.
- This time they should remain between certain inclusive intervals:

1. [0, 50] *(min 0, max 50, both included!)*
2. [10,80]
3. [-20,+20]
4. [-2,+2]

⇒ Hint: use modulo % to stay within a certain range.

⇒ Hint: No idea how to start? check [the help file for rand\(\)](#)!

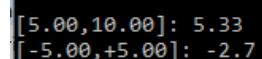


```
[0,50]: 49
[10,80]: 47
[-20,20]: -6
[-2,2]: -2
```

PRINT RANDOM FLOATS IN INTERVAL

- Create / call a third function that generates random floating point values in an interval
- The rand() function only generates whole numbers (integers). Which trick could we use to go from that to a floating point value with 2 decimal places? Try to *think* about how you could solve this!

1. [5.00,10.00]
2. [-5.00,+5.00]



```
[5.00,10.00]: 5.33
[-5.00,+5.00]: -2.7
```

EX02: EXPLORE STRINGS

GENERAL

- Create a new **CONSOLE PROJECT** called **EX02_ExploreStrings** in this lab's folder
- Add `#include <string>` to the includes at the top of your cpp file
- Topics: we will explore `std::string` and `char`, together with **EXPLICIT CONVERSIONS**

ASCII VALUES

Create a function will ask for an ascii value and show the character that goes with it. Choose a clear identifier for it and call it from main.

- ask the user for an ascii value, make sure you store this in an **integer variable**
 - ⇒ next, print both the ascii value and the character that goes with it
 - ⇒ hint: you will need to **convert** your ascii value **to a character**
- example where the user entered 80 :

```
Enter an ASCII value: 80
The ASCII value you entered was 80, which results in character P
```

EMAIL GENERATOR (COMBINING STRINGS AND CHARACTERS)

Create a function that will ask for a first and last name, and generate a howest email address based on the information that was entered.

- ask for the first name and save it in a string, do the same for last name
- create a string variable **email** that combines this input to generate a howest email address
 - ⇒ we want to explore adding strings and characters
 - ⇒ therefore, add the dots as a character '.' to the string!
- example result for input "Noah" and "Fence":

```
Enter your first name: Noah
Enter your last name: Fence
Your email:      Noah.Fence@student.howest.be
```

RHYME GENERATOR (CONVERTING NUMBERS TO STRINGS)

Create a function that will ask for a noun, an adjective and a number. Using this input, it generates the first part of a rhyme.

- ask for a noun and an adjective; save each one in a **std::string** variable
- ask for a number as well, save it in an **integer** variable
- create an extra **std::string** variable called rhyme
 - ⇒ use the compound assignment operator **+=** to add the various parts to the rhyme
 - ⇒ structure using **<>** as placeholders (also see example screenshot further):


```
"<number> <adjective> <noun>, lined up in a row.
One got very tired, so it had to go.
Now there's only <1 less than number> putting on a show."
```
 - ⇒ what happens if you do not convert the number to a string before adding it to the rhyme?
 - ⇒ **Hint:** there is a function in the standard (std::) library that can convert an integer to a string. Try to look it up first; ask a teacher if you cannot find it.
- the variable should contain the entire rhyme, print it: `std::cout << rhyme << std::endl;`
- example result for input "squirrels", "silly" and 10:

```
>> First, give us a noun (plural): squirrels
>> Great! Now pick an adjective: silly
>> Almost there! Enter a number > 2: 10

"10 silly squirrels, lined up in a row.
One got very tired, so it had to go.
Now there's only 9 putting on a show."
```

Any idea how you can print the double quotes "" around the rhyme?

COMBINING INTEGERS AND STRINGS

Create a function that asks for an integer and a floating point value. It adds them together as strings and multiplies them as an int and floating point value.

- ask for an integer but save it in a `std::string` variable
- ask for a floating point value as well, but again save it in a `std::string` variable
- add the two strings together and print the result ("1" + "1" = "11" !)
- convert** the first string value **to an integer**, and the second string value **to a float**
 - ⇒ look up which functions can help you with this; ask a teacher if you cannot find it
 - ⇒ add the two together and show the result (1 + 1 = 2)
- Example result for values "10" and "1.234":

```
Enter an integer: 10
Enter a floating point value: 1.234

"10" + "1.234" = 101.234
10 + 1.234 = 11.234
```

EX03: EXPLORE MATH

-- We will explore existing math functions, first in console, next we use them to draw. All functions needed with info and example code can be found in the [cmath](#) reference --

GENERAL

- Create a new **PROG1 GAME PROJECT** called **EX03_ExploreMath** in this lab's folder
- Topics: we will explore some [cmath](#) functions
- For example, to round up a floating point value:


```
float value{ 1.234f };
int number{ static_cast<int>( ceil( value ) ) }; // ceil, result: 2
```

PRINT A TEST TABLE

Create a function that will print a table with the sin and cos values of some angles (see further).

- WARNING:**
 - To **declare** functions in a game project, add them to the **game.h** file! (Find the region for your own code).
 - Next, implement your functions in the **game.cpp** file (at the bottom).
- in the function, create a **local float variable pi** which contains a value for pi, eg. 3.14159
- create a second float variable called angle, which will contain an angle in **radians**
 - **Hint:** all functions you need can be found in the [cmath](#) reference
 - 1. initialize the angle with **0**, and print its cos and sin value
 - 2. change the angle's value to **pi** and print again
 - 3. change it to a full circle (**2 times pi**) and print again
 - Call the function from **Draw()** first; the result appears in the console window behind the game window. Next, only call it from **Start()** instead. What difference do you notice?

- check the screenshots below: the left shows the scientific notations of some decimals; fix this by rounding the values to get the result on the right:

<pre> radians: 0 sin: 0 cos: 1 radians: 3.14159 sin: -8.74228e-08 cos: -1 radians: 6.28319 sin: 1.74846e-07 cos: 1 </pre>	➔	<pre> radians: 0 sin: 0 cos: 1 radians: 3.14159 sin: -0 cos: -1 radians: 6.28319 sin: 0 cos: 1 </pre>
---	---	---

EX04: AWAITS HUMAN REVIEW

Unzip the **given cmake project** with the same name as the title above.

You have been given a **piece of AI-generated C++ code**. Your task is to give it a code review and identify mistakes, bad practices, guideline violations, and logic errors.

Your goal

- Use the [Core guidelines & naming conventions](#) section at the start of this document as reference.
 - The new ones introduced in this lab are indicated in **blue**.
- For every guideline violation, **mention the corresponding guideline code** (e.g. ES.23, NL.5).
- **Write your findings as comments** above or next to the relevant line(s) of code.
- Always **explain in short** why the code is problematic, not just what is wrong.
- For each issue you find, comment out original code and **write the corrected version underneath it**.

IMPORTANT

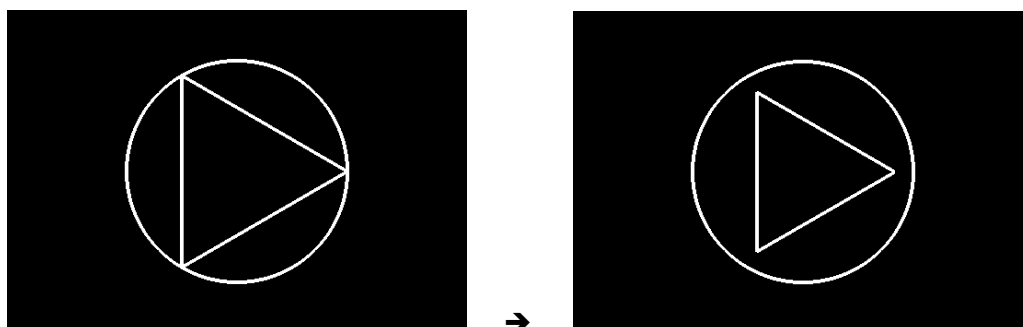
IF YOU ARE PLANNING TO ASK AN AI TO REVIEW THE AI-GENERATED CODE FOR YOU, PLEASE SKIP THIS ASSIGNMENT ENTIRELY. YOU'LL SAVE:

- *YOUR TIME,*
- *OUR TIME,*
- *A FEW SERVERS,*
- *AND POSSIBLY AN ENTIRE FOREST WORTH OF ELECTRICITY.*

More importantly, you'll miss the actual learning goal: developing the ability of critical thinking. In a few weeks, you won't be graded on your ability to ask an AI for answers. You'll be graded on your ability to review code yourself. This will be done in a **closed-book** environment, without AI, notes, or online resources.

EX03: HANDS-ON DEMO: USING TRIGENOMETRY TO DRAW

Open your **EX03_ExploreMath** project folder; we will continue working in it.



- create a function to draw a “play button” and call it from draw
- draw a circle with a thick stroke, make sure to use variables for both the position and the radius
- use those variables in combination with trigonometry to draw the 3 lines as above in the left screenshot; which formulas do you need?
- next, try to apply the necessary changes to go from the result in the left screenshot to that in the right

EX05: DRAWING WITH BASIC SHAPES

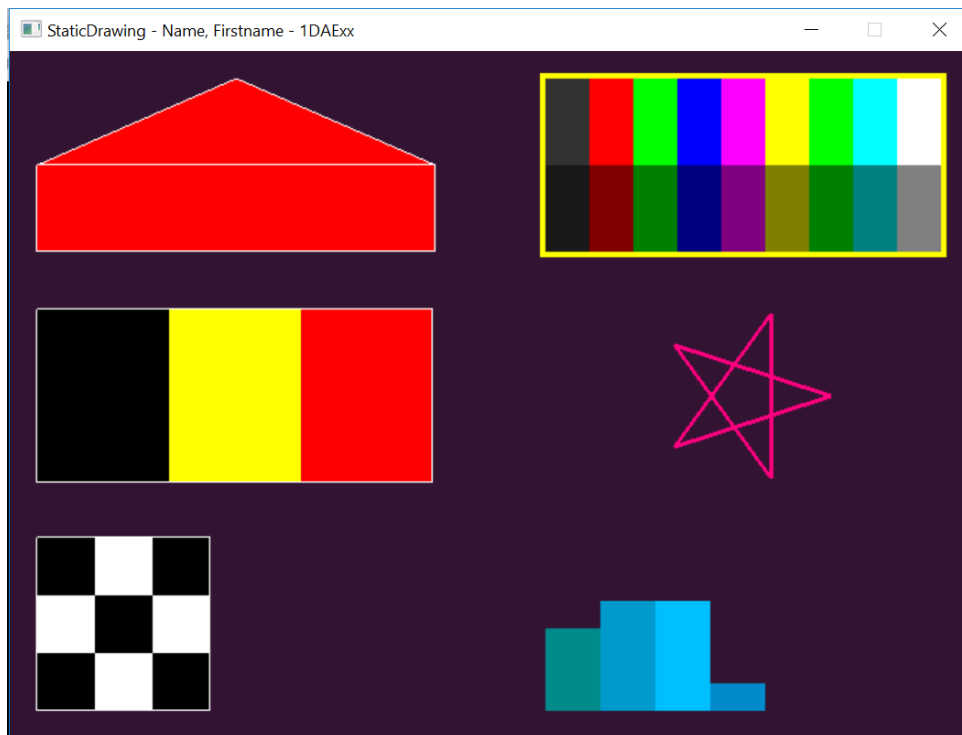
*If you already have programming experience, you can skip the exercises that might seem repetitive and spend more time on the more challenging and the free exercise described at the end of this document. However, do the **pentagram** and **columns graph** exercises as you’ll animate them in one of the later assignments.*

*Be careful: do not overestimate your programming skills!
This is a common student mistake.*

GENERAL

Create a new **Prog1 Game Project** called **EX05_BasicDrawing** in this week’s project folder. Adapt the **TITLE** of the window.

Then **make the exercises as described further in this document**. In the end you should have a window that looks like this:



MAKE THE 6 DRAWINGS

Use **VARIABLES** whenever you can, make the figures easily adjustable by using variables and expressions instead of hard coded numbers.

Make your code easy to read – for yourself and your teacher – and split it into functions. As a function performs a task use a **VERB** in its name e.g.:

```
void DrawHouse( );  
void DrawFlag( );  
void DrawCheckerPattern( );  
void DrawColorBand( );  
void DrawPentagram( );  
void DrawColumnChart( );
```

These functions should be declared in the Game.h header file where it is mentioned as comment. The definition is in the Game.cpp file below all the other functions.

The drawing should contain following parts:

1. House (do this as the last one)
2. Belgian flag
3. Checker pattern
4. Colored band
 - Draw 9 rectangles that are 4 times higher than they are wide in different colors. Do not forget the yellow border.
 - After that, darken the lower half by drawing a filled dark rectangle using a color that has an alpha component that is less than 1.
5. Pentagram: use trigonometry to determine the coordinates of 5 points situated on a circle and then draw lines between these points.
6. Column chart of game playing percentage per following age groups:
 - [0,20]
 - [20,40]
 - [40,60]
 - Older than 60

MAINTAINABLE AND READABLE CODE

Ask yourself the question what code you need to modify when you want to change following values:

1. House : position, width and height
2. Flag: position, width and height
3. Checker pattern: position and size
4. Colored band: position and width
5. Pentagram: center and radius
6. Column chart: position, column width, percentages

What changes did you need to make?

1. Only initialization of variables?
2. The drawing operations too?

When you need to change the drawing operations as well, then **ADAPT YOUR CODE** so that changing the drawing code is no longer necessary when another position or size is required.

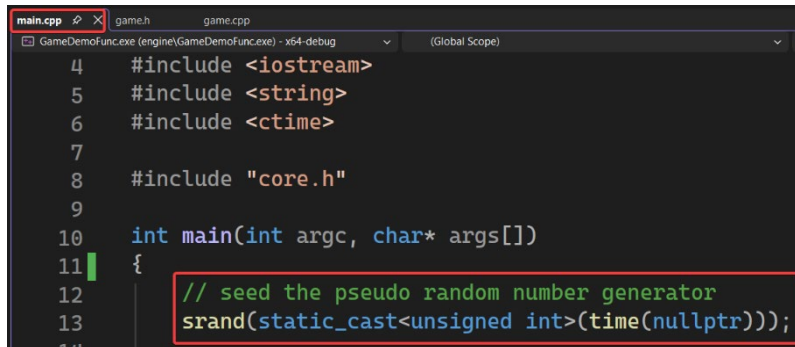
EX06: RANDOM RECTANGLES

Create a new Prog1 Game Project with name **EX06_RandomRects** in your lab folder. Adapt the window title.

WARNING 1: If you are sensitive to high frequency screen flashes, DO NOT MAKE THIS EXERCISE.

WARNING 2: For **game project (graphical) applications** the seeding srand already happens in the main function

- do **not** call it again in draw as it will slow down the draw loop significantly!



```
4 #include <iostream>
5 #include <string>
6 #include <ctime>
7
8 #include "core.h"
9
10 int main(int argc, char* args[])
11 {
12     // seed the pseudo random number generator
13     srand(static_cast<unsigned int>(time(nullptr)));
14 }
```

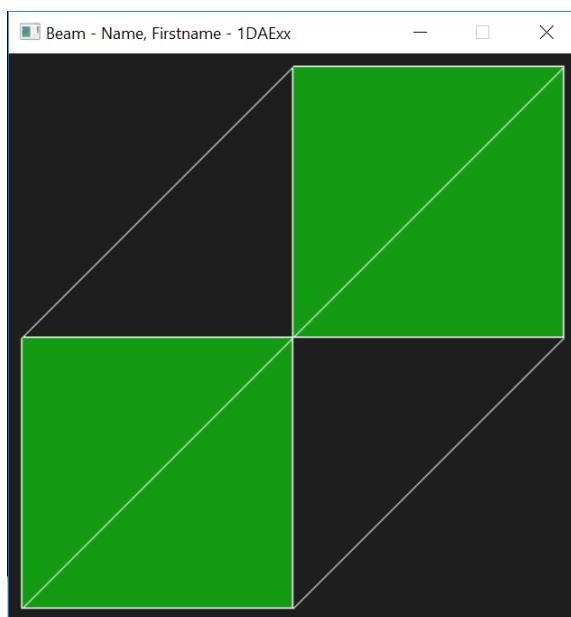
Generate a rectangle with rand color, random size and position:

- only **one rectangle** is drawn every draw loop
 - by using random size and position, it appears to be animating the rectangles
 - Because of the high framerate, a new rectangle will be drawn each frame.
- The rectangle must **stay inside the window** and keep a distance of 10 pixels from the edges.
 - **Hint 1:** make sure the rectangle size is always at least 1px!
 - **Hint 2:** comment out the ClearBackground() instruction from Draw to check if distance works.
 - **Hint 3:** position and size will be related to each other...

EX07: BEAM

Create a new game project with name **Beam** in this week's lab folder.

- Adapt the window title.
- Write the code to get the image below
 - take into account the **restrictions** as listed below.



RESTRICTIONS

Comply with following restrictions:

- The application window size is 420 x 420 pixels
- The background is black.
- The squares are green and have a white border and their vertices are connected with white lines.
- Use a **local** variable to hold the size of the squares, the value is 200.
- Use a **local** variable to hold the distance to the window-border, the value is 10.
- You are allowed to use only the variables `g_WindowWidth`, `g_WindowHeight`, the two variables described above; **other** variables (or magic numbers) are **NOT** allowed (except for 0).
 - o **Note:** by using `g_WindowWidth` and `g_WindowHeight`, we can always resize or window and everything should just move along with it.

CHANGE THE WINDOW SIZE

Changing the window size to 550 x 450 should result in this drawing without changing any other code. Verify this in your code.



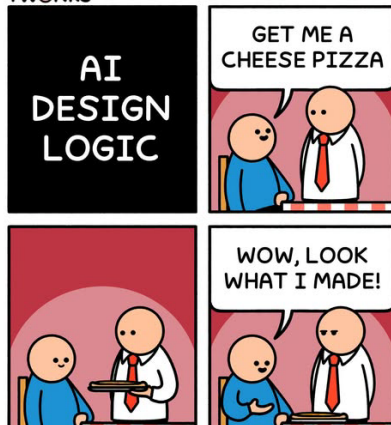
EX08: CODING CRAFTSMANSHIP CHALLENGE

DO NOT SKIP THIS EXERCISE; EXPERIMENTING WITH CODE IN A SELF-MADE PROJECT IS BY FAR THE BEST WAY TO MASTER PROGRAMMING!

INSTRUCTIONS

Create either a **Prog1 Game Project** called **CodingCraft02**.

TWONKS



Create a **small original application** that demonstrates the concepts we've covered so far.

- ✓ Create **your own** original project.
- ✓ **Only** use **concepts that have been covered** in labs already.
- ✓ Write code you can explain yourself.
- ✗ **No** AI-generated code.
- ✗ **No** copied solutions from the internet.
- ✗ **No** copying another student's work.

For example, this week: draw your favorite logo or character using basic shapes or find inspiration by searching for daily minimal in Google images.

WALL OF FAME

Every week, some projects will earn a place on the lab's Wall of Fame.

A project does not need to be complex to deserve a spot. Original ideas, clever concepts, fun themes, polished outputs, and well-written code are just as valuable. Projects are shown **anonymously** so you can choose whether you reveal yourself as the creator or not.

The **goal** is to **learn, experiment, and become a better programmer**. The Wall of Fame only works if every project genuinely reflects your own work. **Let your imagination run wild and work on a programming project YOU love!**

SUBMISSION INSTRUCTIONS

You must submit **two files**:

- A **zip file** containing your **complete lab folder** of this week with all exercises.
- **CodingCraft02.zip** containing only your Coding Craftsmanship Challenge project.

BEFORE ZIPPING

For every project:

- **Close** Visual Studio.
- Delete the **out** folder.
- Delete the hidden **.vs** folder.

SUBMIT

- Upload **both files** with the correct assignment before the start of next week's first lab.
- Make sure to complete the **quiz**.
- *These steps are required to unlock the next lab assignment!*

 Your submission may be discussed during a code review. Make sure you understand and can explain your own code.

REFERENCES

STRUCTURE OF A PROGRAM

http://www.cplusplus.com/doc/tutorial/program_structure/

COUT, CIN, ENDL, STD, USING

A first look at cout, cin, endl, the std namespace, and using statements.

<http://www.learncpp.com/cpp-tutorial/1-3a-a-first-look-at-cout-cin-endl-namespaces-and-using-statements/>

LITERALS

<http://www.learncpp.com/cpp-tutorial/28-literals/>

OPERATOR PRECEDENCE AND ASSOCIATIVITY

<http://www.learncpp.com/cpp-tutorial/31-precedence-and-associativity/>

COMPILING AND LINKING

<http://www.cprogramming.com/compilingandlinking.html>